

Decoupled Early Exits for Task-Dependent Compute Allocation in Flow-Matching VLAs

Riccardo Andrea Izzo¹, Rimvydas Rubavicius², Gianluca Bardaro¹,
Subramanian Ramamoorthy², Matteo Matteucci¹ and Alessandro Suglia²

Abstract—Flow-matching Vision-Language-Action (VLA) models have emerged as a potential solution for generalist robot control, designed by combining a pretrained Vision-Language Model (VLM) backbone with an action expert that generates continuous robot actions. While these models exhibit impressive capabilities, due to their very high number of parameters, their computational requirements are often prohibitive for robotics control. To mitigate these inefficiencies, existing methods predominantly skip VLM backbone layers with early exits or reduce denoising steps, while leaving action expert depth untouched. We propose a framework that exposes backbone depth V , action expert depth A , and denoising steps D as three jointly configurable compute axes in a VLA. Starting from a pretrained VLA, we attach lightweight Exit Transformers (ET) at intermediate depths in both the backbone and the action expert, trained to distil the last layer of the policy into each exit. Furthermore, we introduce a KV Cache synthesis mechanism that manages the missing keys and values of the skipped backbone layers, allowing the action expert to exit deeper than the backbone. Finally, we show that the optimal compute budget is task-dependent, with different tasks benefiting from different axes and depths. Notably, our method does not require training the original policy from scratch, and for each exit, it increases the number of parameters by only 2.1% for SmolVLA and 4.1% for $\pi_{0.5}$. We validate our approach across two flow-matching VLAs (SmolVLA, $\pi_{0.5}$) and two benchmarks (LIBERO, Meta-World), revealing complementary effects: V and A respectively reduce FLOPs and latency, while D improves both. Our joint configurations (V, A, D) reduce latency by 79.2% and computation (FLOPs) by 31.8%, while improving mean success rate by 5.6%.

I. INTRODUCTION

Vision-Language-Action (VLA) models map visual observations, language instructions, and current robot state directly to continuous control actions [1,2]. VLAs are composed of a Vision-Language Model (VLM) that encodes the observations and instructions into a latent representation, and an action expert that generates a set of continuous actions through the denoising process conditioned on this latent representation. Modern state-of-the-art VLAs use a flow-matching objective for action generation to generate smooth and precise trajectories, iteratively refining the action predictions from noise [3].

¹Riccardo Andrea Izzo, Gianluca Bardaro and Matteo Matteucci are with the Department of Electronics, Informatics, and Bioengineering, Politecnico di Milano, Milan, Italy {riccardo.izzo, gianluca.bardaro, matteo.matteucci}@polimi.it

²Rimvydas Rubavicius, Subramanian Ramamoorthy and Alessandro Suglia are with the School of Informatics, University of Edinburgh, Edinburgh, UK {rimvydas.rubavicius, s.ramamoorthy, asuglia}@ed.ac.uk

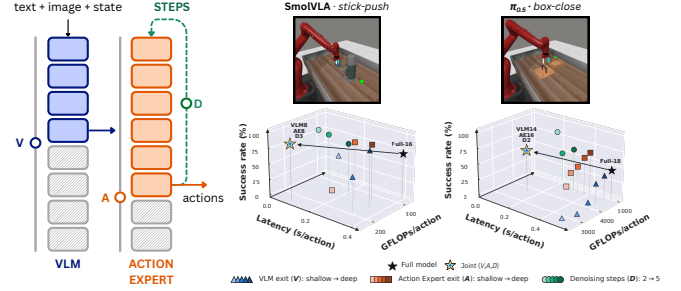


Fig. 1. (V, A, D) compute axes on a flow-matching VLA. Left: VLM depth V , the action expert depth A and the denoising steps D . Right: multi-objective Pareto curves for two exemplary tasks on Meta-World MT50 [4], *stick-push* and *box-close*, across SmolVLA and $\pi_{0.5}$. Our joint configuration outperforms the base policy in latency, FLOPs and success rate.

While VLAs exhibit increasingly strong generalisation capabilities, they remain computationally expensive, with inference times that are often impractical for real-time robot control frequency. This is particularly relevant for flow-matching VLAs, with the action head that requires multiple denoising steps to generate a single action chunk.

To this end, previous methods have proposed different solutions to reduce the inference cost of VLAs, such as layer pruning [5,6], early exits [7,8], or caching [9]. Almost all of them target the vision-language backbone, which holds most of the parameters and therefore dominates the FLOPs count, while others adaptively adjust the number of denoising steps [8,10]. Regarding this, the action expert has been largely overlooked, despite being the component that runs at every denoising step and therefore dominates latency. To the best of our knowledge, many techniques have been proposed to reduce the *depth* (i.e., the number of layers) of a vision-language backbone, but none has been applied to the action expert depth as an independent compute axis, and no study has measured the actual trade-off in how these axes interact.

We argue that early exit applies to the action expert as much as to the vision-language backbone, and that the two must be exposed jointly rather than used in isolation: early exiting the VLM backbone saves a cost paid once per action chunk, whereas exiting the action expert removes layers that would otherwise be executed at every denoising step. We further show that the optimal budget is task-dependent, since different tasks have different semantics as well as different requirements in terms of precision and speed, and therefore the optimal compute budget may vary across tasks. Fig. 1 gives a glimpse of this on two Meta-World tasks with

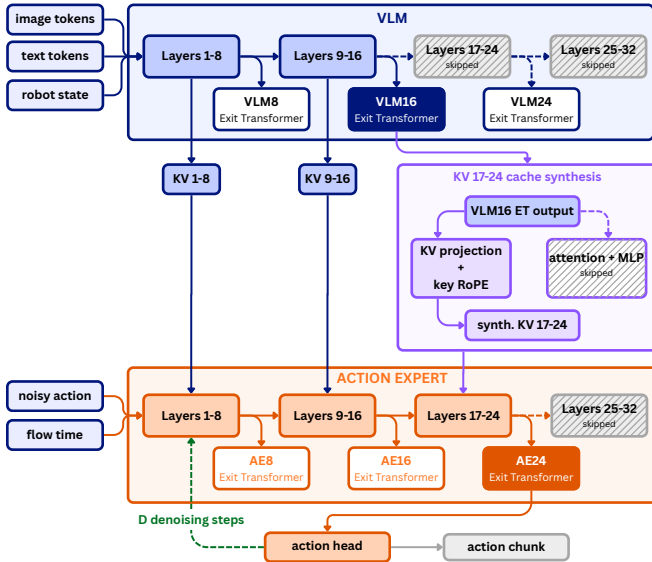


Fig. 2. **Three-axis compute budget** (V, A, D). Exit Transformers expose intermediate depths in the VLM (V) and the action expert (A), while D sets the number of denoising steps. An early exit leaves the prefix KV cache of layers $V < i \leq A$ empty, and those slots are synthesised from the ET output. Shown: SmolVLA with $(V, A, D) = (16, 24, D)$.

opposite requirements. For *stick-push* VLA requires more fine-grained dexterity, and is therefore limited by the action expert while tolerating a shallow backbone. On the other hand, *box-close* relies on perceiving the lid relative to the box, and is therefore limited by the VLM backbone while tolerating less depth in the action expert.

In this paper, we present a framework that exposes all three axes on a flow-matching VLA: backbone depth V , action expert depth A , and the number of denoising steps D . Lightweight Exit Transformers (ET) are attached at intermediate depths of both the VLM backbone and the action expert. Each ET is initialised as a copy of the transformer block at the corresponding depth, and is trained by distilling the last layer of the frozen policy into it, so that the pretrained action head is reused unchanged and every exit stays in the original action space. The VLM backbone and the action expert are never updated, which preserves the exact behaviour of the full VLA and makes the optimal compute configuration a runtime choice rather than a separate model. We also identified a bottleneck that arises when a backbone exit is used: the deeper expert layers still need the prefix keys and values that the skipped layers never computed, so we refill those cache slots by re-projecting the exited prefix with each skipped layer’s own key and value matrices, at the cost of two projections instead of a full block. This further improves the savings of a backbone exit, without requiring additional training or inference cost. Finally, the number of denoising steps (D) is the third axis and requires no training. Fig. 2 provides an overview of the framework.

Extensive evaluation shows that joint configurations reduce latency by 79.2% and FLOPs by 31.8%, while improving mean success rate by 5.6%. On LIBERO and Meta-World benchmarks and across SmolVLA and $\pi_{0.5}$, our framework

improves the Pareto frontier (success rate vs. latency/FLOPs) over full compute base models.

Our contributions are:

- A framework for flow-matching VLAs, with early exits on both the VLM backbone V and the action expert A , paired with a KV cache synthesis mechanism and a variable number of denoising steps D , while adding a negligible number of parameters and avoiding training from scratch.
- An evaluation of the three axes on two flow-matching VLAs (i.e., SmolVLA and $\pi_{0.5}$) and two benchmarks (i.e., LIBERO and Meta-World), measuring trade-offs and the marginal effect of each against a full-compute model in terms of success rate, FLOPs, and latency.
- A task analysis, showcasing that different tasks benefit from different axes, and that the optimal compute budget is task-dependent.

For reproducibility, we release our code and models¹.

II. RELATED WORK

A. Vision-Language-Action Models

Vision-Language-Action (VLA) models have emerged as a promising approach for generalist robotics policies by combining language, vision, and action prediction in an end-to-end framework. These policies are usually trained on massive multimodal datasets comprising images and language instructions paired with collected robot trajectories. Since the emergence of VLAs, early approaches such as RT-2 [11] and OpenVLA [12] have built on top of pre-trained VLMs, processing image observations and text instructions to produce discretised actions. Due to the precision limitations of discretisation, subsequent models such as π_0 [13] transitioned to flow-matching networks to better capture the multimodal nature of robot trajectories. Furthermore, to address the computational requirements of previous VLAs, SmolVLA [14] and TinyVLA [15] proposed compact and efficient backbones without sacrificing performance.

B. Efficient and Adaptive VLA Inference

Among efficient inference approaches, one of the most effective and widely studied is early exiting [16]. This technique minimises computation by attaching intermediate predictors and terminating inference once a sufficiently reliable output is available. This paradigm was established for CNNs by BranchyNet [17], and later extended for Transformers by DeeBERT [18]. Within the field of robotics, DeeR-VLA [7] applied this concept to VLAs by adaptively terminating the VLM backbone based on the current situation or computational requirements. Concurrently, AC²-VLA [19] and MoLE-VLA [6] directly skip selected backbone layers. An orthogonal direction exploits temporal and structural redundancy: VLA-Cache [9] reuses visual token features across control timesteps, while EfficientVLA [5] uses language layer pruning, visual token selection, and diffusion

¹<https://github.com/esgi-research-group/ee-vla>

caching. Recent methods also target iterative action generation. For instance, CEED-VLA [20] applies consistency distillation and early termination to the Jacobi decoding iterations of autoregressive VLAs. For flow-matching policies, SnapFlow [10] distills generation to a single forward pass, while A1 [8] exits when consecutive action predictions agree and uses inter-layer truncated flow matching to warm-start denoising at the next candidate depth. In A1, the backbone and the action expert are truncated together at the same layer, coupling their depth. Our formulation instead exposes the backbone depth V and action expert depth A as independent axes through the use of decoupled early exits, alongside the number of denoising steps D . We argue that this decoupling matters because V and A have different computational roles, with the backbone being evaluated once per action chunk, while the action expert is evaluated at every denoising step. Our formulation takes advantage of a KV cache synthesis mechanism, allowing configurations with the action expert that exits deeper than the backbone ($A > V$). Moreover, while A1 is developed and trained around a specific Molmo-based architecture, our method is agnostic to flow-matching VLAs, and requires distilling early exits into frozen pretrained policies, thereby avoiding VLA pretraining. We demonstrate this on two different VLAs (i.e., SmoVLA and $\pi_{0.5}$), and systematically characterise the individual and joint trade-offs of V , A , and D .

III. METHOD

A. Preliminaries

Formally, the goal of a VLA is to learn a policy $\pi(\mathbf{A}_t \mid \mathbf{o}_t, l, \mathbf{q}_t)$ that, at each timestep t , maps a list of k camera observations $\mathbf{o}_t = [o_t^1, \dots, o_t^k]$, a language instruction l , and a proprioceptive state $\mathbf{q}_t \in \mathbb{R}^{d_q}$ (e.g., joint angles) to a chunk of H future actions $\mathbf{A}_t = [a_t, a_{t+1}, \dots, a_{t+H}]$, $a \in \mathbb{R}^{d_a}$.

a) Architecture: Flow-matching VLAs realise π with two components: a pretrained VLM backbone and an action expert, each implemented as a Transformer stack of N layers indexed by $i \in \{1, \dots, N\}$ [14, 21]. At every layer, the token sequence splits into two contiguous blocks. The *prefix* carries the multimodal observation including the visual tokens extracted from $\mathbf{o}_t$ by the VLM vision encoder, together with the tokenised instruction l , for a total of M tokens. The *suffix* includes a linear projection of the proprioceptive state $\mathbf{q}_t$ and the H noisy action tokens $\mathbf{A}_t^\tau$ at flow-matching time τ .

We define $\mathbf{P}_i \in \mathbb{R}^{M \times d_b}$ and $\mathbf{S}_i \in \mathbb{R}^{H \times d_e}$ for the hidden states output by layer i of the backbone and of the expert, respectively, where d_b, d_e are the two model widths. When $i = 0$, $\mathbf{P}_0$ and $\mathbf{S}_0$ correspond to the input embeddings. The action expert is conditioned on the VLM backbone through the per-layer prefix keys and values. Since the backbone and expert have the same depth, the suffix queries at layer i attend to the keys and values computed at that same layer from $\mathbf{P}_{i-1}$. The conditioning context is therefore the full set of per-layer prefix states, $\mathbf{c} \triangleq \{\mathbf{P}_i\}_{i=0}^{N-1}$, rather than a single pooled vector. Following the default configurations of SmoVLA [14] and $\pi_{0.5}$ [21], we assume the backbone and expert have the same depth N , so that expert layer i reads

from backbone layer $i - 1$. At full depth, the action head h maps the last H suffix tokens of layer N to the predicted velocity field, $v_\theta(\mathbf{A}_t^\tau, \tau, \mathbf{c}) \triangleq h(\mathbf{S}_N)$.

b) Training and inference: The policy is usually trained by imitation learning on a dataset of tuples $(\mathbf{o}_t, l, \mathbf{q}_t, \mathbf{A}_t)$, minimising the discrepancy between predicted and ground-truth actions. Specifically, we use the conditional flow-matching loss [3]:

$$\mathcal{L}(\theta) = \mathbb{E} \|\mathbf{v}_\theta(\mathbf{A}_t^\tau, \tau, \mathbf{c}) - \mathbf{u}(\mathbf{A}_t^\tau \mid \mathbf{A}_t)\|^2, \quad (1)$$

where, following the convention of $\pi_{0.5}$ [21], $\tau \in [0, 1]$ is the flow-matching time, $\mathbf{A}_t^\tau = \tau\epsilon + (1-\tau)\mathbf{A}_t$ is the noisy chunk built from $\epsilon \sim \mathcal{N}(0, \mathbf{I})$, and $\mathbf{u}(\mathbf{A}_t^\tau \mid \mathbf{A}_t) = \epsilon - \mathbf{A}_t$ is the target vector field. At inference time, the chunk is generated by integrating the learned field from $\mathbf{A}_t^1 \sim \mathcal{N}(0, \mathbf{I})$ down to $\tau = 0$ in D Euler steps of size $\delta = 1/D$:

$$\mathbf{A}_t^{\tau-\delta} = \mathbf{A}_t^\tau - \delta \mathbf{v}_\theta(\mathbf{A}_t^\tau, \tau, \mathbf{c}), \quad (2)$$

after which $\mathbf{A}_t^0$ is executed on the robot. Each of the D steps re-runs the N expert layers over the suffix, while $\mathbf{c}$ is computed once.

B. Three-Axis Compute Budget

We reduce inference cost along three axes and define the resulting compute budget as (V, A, D) . V and A select the early exit depths of the VLM and action expert, respectively, while D controls the number of denoising steps. Accordingly, $\mathbf{P}_V$ is the prefix state at depth V and $\mathbf{S}_A$ is the suffix state at depth A , with $V = A = N$ corresponding to full depth execution along both axes.

Our goal is to preserve success rate while reducing the number of layers executed, and therefore the FLOPs and latency of the policy. However, since the original policy was trained to predict only from its final layer, intermediate states cannot be used directly, as the expert expects $\mathbf{P}_N$ and the action head h expects $\mathbf{S}_N$. Following FREE [22], we implement V and A by attaching to each candidate depth a lightweight *Exit Transformer* (ET), a single Transformer block initialised from the layer at that depth. Unlike FREE, we align the exits through direct supervision rather than adversarial training, as described in Section III-D.

VLM ET (V): An early exit in the VLM consists of a Transformer layer whose architecture matches that of a layer in the VLM backbone. At each candidate depth V , we attach an ET and apply it to the prefix with bidirectional self-attention. The output of the ET, namely $\tilde{\mathbf{P}}_V = \text{ET}_V(\mathbf{P}_V)$, substitutes the layers that were not executed, so that the frozen action expert is conditioned on a representation approximating the prefix at full depth $\mathbf{P}_N$.

Action Expert ET (A): Similarly, an action expert ET consists of a single Transformer block that matches the architecture of the original action expert layer, using causal self-attention over the H action tokens. However, its objective is different from that of a VLM ET. Instead of producing an intermediate representation that is consumed by another layer, it should directly yield the velocity from the suffix $\mathbf{S}_A$ produced at the selected depth A , $v_\theta^{(A)} =$

TABLE I
TRAINING HYPERPARAMETERS OF THE ET FOR EACH CHECKPOINT.

Checkpoint	N	Exits	LR	Warm.	Batch	Steps
<i>HuggingFaceVLA/smolvla_libero</i>	32	{8, 16, 24}	10^{-4}	1000	8	31,184
<i>lerobot/smolvla_metaworld</i>	16	{4, 8, 12}	10^{-4}	1000	8	25,600
<i>lerobot/pi05_libero_finetuned_v044</i>	18	{6, 9, 12, 14, 16}	10^{-4}	500	8	15,592
<i>tiantianx/pi05_metaworld</i>	18	{6, 9, 12, 14, 16}	10^{-4}	1000	8	25,600

$h(\text{ET}_A(\mathbf{S}_A))$. The remaining layers of the expert are never executed, saving a fraction $(N - A)/N$ of the suffix pass at each of the D denoising steps.

Denoising steps (D): As defined in Eq. 2, a denoising step is a single evaluation of the velocity field that advances the noisy chunk by δ towards the final trajectory. This is repeated D times, defining the number of steps in the Euler integration. Therefore, this third axis of our compute budget does not require any trainable parameters. The step count is a parameter that trades integration accuracy against compute and can be changed at inference time without retraining.

C. KV Cache Synthesis

At layer i , the action expert attends to the keys ($\mathbf{K}_i$) and values ($\mathbf{V}_i$) computed from the prefix at the corresponding layer i . Both $\mathbf{K}$ and $\mathbf{V}$ are computed by the VLM backbone in a single prefix pass and stored in a cache that is later read at each denoising step. A VLM early exit breaks this correspondence. Building the prefix through depth V only fills the KV cache slots up to that depth, so every action expert layer running deeper (i.e., $i > V$) finds its slot empty. Prior works with LLMs either propagate the state of the exit layer to all deeper layers, projecting it with each layer’s own key and value matrices [23,24], or recompute the missing entries when required [25]. We follow state propagation [23,24], but propagate the ET output $\tilde{\mathbf{P}}_V$ instead of the prefix state $\mathbf{P}_V$ at the exit layer. Every backbone layer i with $V < i \leq A$ projects it with its own key and value learnable matrices $\mathbf{W}_{k,i}$ and $\mathbf{W}_{v,i}$:

$$\mathbf{K}_i = \text{RoPE}(\mathbf{W}_{k,i}\tilde{\mathbf{P}}_V), \mathbf{V}_i = \mathbf{W}_{v,i}\tilde{\mathbf{P}}_V, \tilde{\mathbf{P}}_V = \text{RMSNorm}_i(\tilde{\mathbf{P}}_V) \quad (3)$$

so that the action expert finds the KV entries for all of its A layers. In practice, since the prefix is fixed for the whole action chunk and the outputs of the skipped layers are never read, we synthesise all the missing entries at once in the prefix pass, without running those layers. As illustrated in Eq. 3, each skipped layer only runs its RMSNorm, two projections, and key rotation, while its attention, output projection, and feed-forward are skipped, leading to significant speedups outlined in Table III. This synthesis adds no approximation beyond state propagation itself, as the synthesised entries are identical to those obtained by running each skipped layer on $\tilde{\mathbf{P}}_V$. At the end, the prefix pass costs V layers, the ET and the projections of Eq. 3, instead of N full layers.

D. Training

We train the early exits by distillation, with the last layer of the corresponding module as teacher ($\mathbf{P}_N$ for the VLM backbone and $\mathbf{S}_N$ for the action expert) and each ET as

student, thus aligning their representations. The backbone and the action expert are never updated, so that the behaviour of the original policy at full compute is not affected. We denote by v_θ the full-depth velocity, and by $v_\theta^{(V)}$ and $v_\theta^{(A)}$ the velocity when the backbone exits at depth V or the expert at depth A , respectively. Since the action expert and VLM ETs produce different outputs, the two are supervised by different training objectives.

VLM ET: A VLM ET is trained to output an intermediate representation that every action expert layer deeper than V will consume. The loss for each VLM exit at depth V minimises:

$$\mathcal{L}_V = \lambda_a \text{MSE}(v_\theta^{(V)}, v_\theta) + \lambda_d d(\tilde{\mathbf{P}}_V, \mathbf{P}_N). \quad (4)$$

The first term is a velocity distillation that minimises the Mean Squared Error (MSE) between $v_\theta^{(V)}$ and v_θ . The second term is a feature distillation that instead minimises the cosine distance d between the prefix $\tilde{\mathbf{P}}_V$ and the one at full depth $\mathbf{P}_N$. The first term, therefore, measures the effect of the intermediate representation on the generated action, while the second supervises the alignment of the representations.

Action expert ET: An action expert ET is instead trained to output the velocity directly. Since no deeper layer consumes its representation, the feature distillation term is dropped in favor of a standard flow-matching loss term. The loss for each action expert exit at depth A minimises:

$$\mathcal{L}_A = \lambda_f \text{MSE}(v_\theta^{(A)}, u) + \lambda_a \text{MSE}(v_\theta^{(A)}, v_\theta). \quad (5)$$

The first term is the flow-matching objective of Eq. 1, while the second term is the same distillation term as in Eq. 4.

We set λ_f , λ_a and λ_d with a grid hyperparameter search, retaining the configuration with the highest success rate on LIBERO and Meta-World at the corresponding exit depths. This yields $\lambda_a = 1$, $\lambda_d = 0.2$ for Eq. 4 and $\lambda_f = 1$, $\lambda_a = 0.5$ for Eq. 5. Exits at different depths are trained jointly for each axis by averaging their losses, so that the objective does not depend on the number of enabled exits, and each ET receives gradients only from its own term. Each ET is trained against a full depth counterpart on the other axis, and the two are composed only at inference time. Finally, the addition of the ETs does not significantly increase the overall model size. For SmolVLA, an action expert ET adds 3.1M parameters and a VLM ET adds 9.8M, against the 605M of the frozen model. For $\pi_{0.5}$, the same ETs add 23.6M and 110.1M parameters to a total of 3.3B.

IV. EXPERIMENTS

A. Experimental Setup

Benchmarks: We evaluate our method on two benchmarks: LIBERO [26], a large-scale benchmark for long-horizon robot manipulation, and Meta-World *MT-50* [4], a suite of 50 robotic manipulation tasks across four levels of difficulty (i.e., easy, medium, hard, and very hard).

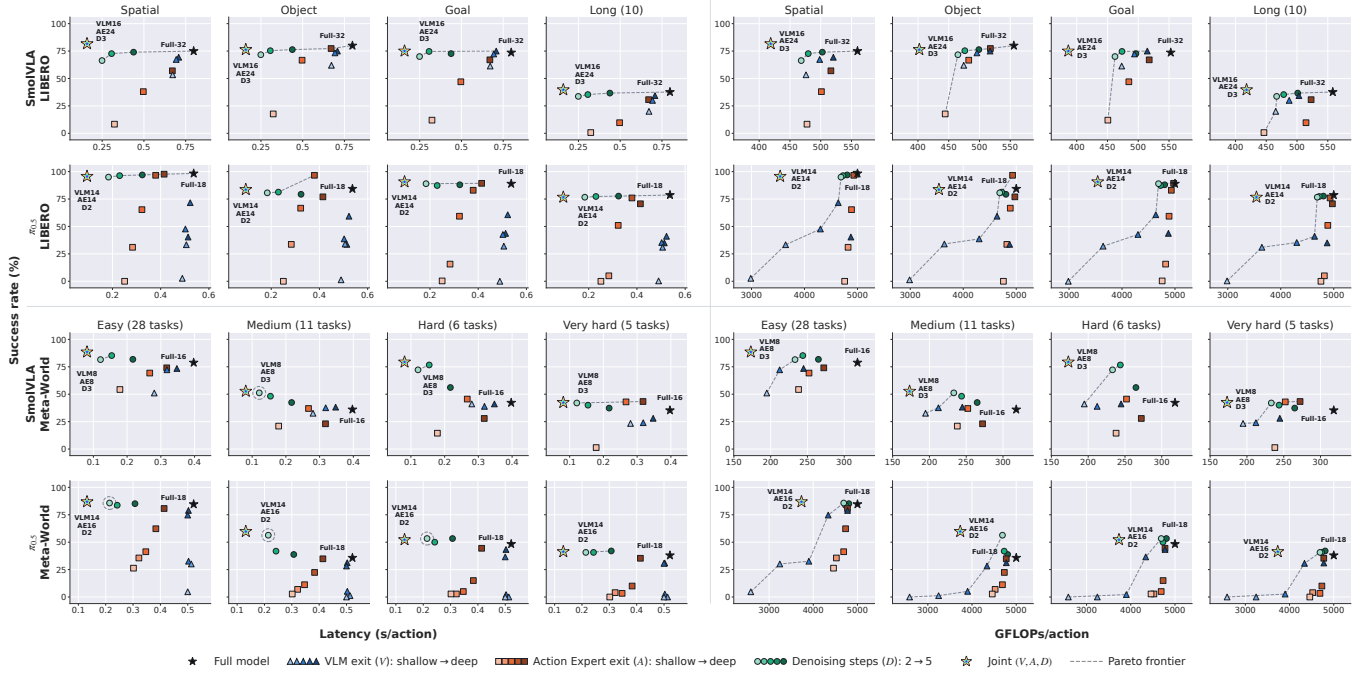


Fig. 3. **Pareto curves on three axes.** Success rate (SR) against latency (left) and FLOPs (right) for SmolVLA and $\pi_{0.5}$, across LIBERO (top) and Meta-World (bottom). Each point changes one axis of the compute budget (V, A, D) while the other two are fixed, with the full policy as reference. All VLM exits (V) points include the KV cache synthesis of Eq. 3; rainbow stars denote the optimal joint configurations (V, A, D) reported in Table II. Every point is averaged over 30 episodes per task, so $N = 300$ for LIBERO suite and $N = 840/330/180/150$ for the four Meta-World difficulty groups, which corresponds to a mean 95% confidence interval of $\pm 2.67\%$ on LIBERO and $\pm 3.33\%$ on Meta-World.

Datasets and models: We train the ETs within the LeRobot [27] framework on the datasets corresponding to the benchmarks, namely LIBERO (*lerobot/libero*) and Meta-World (*lerobot/metaworld_mt50*). For both SmolVLA and $\pi_{0.5}$, we start from a checkpoint already fine-tuned on the dataset, which is kept frozen to train only the ETs. We report all the hyperparameters in Table I.

Evaluation Metrics: We report success rate (SR), the fraction of successful episodes over all tasks of a suite; FLOPs, measured with the PyTorch profiler on one action-generation call (backbone plus the full denoising loop); and latency, the wall-clock time of that call, averaged over the evaluation. Latency is measured at batch size one to avoid batched throughput. All training and evaluation runs are performed on a single NVIDIA A100 with 40 GB of VRAM.

Specifically, we aim to address the following research questions (RQ):

- **RQ1 (Axis roles and composition):** How do V, A and D individually trade success rate against latency and FLOPs, and can their complementary effects be composed?
- **RQ2 (Task dependence):** Is the optimal compute budget task-dependent? Do different tasks and policies favour different axes and depths?
- **RQ3 (Early Exits):** Can early exits be applied to the action expert and are they beneficial over layer truncation? Is the KV cache synthesis effective in allowing the action expert to run deeper than the backbone?

TABLE II
BASE AND JOINT CONFIGURATIONS. JOINT CONFIGURATIONS (V, A, D) OUTPERFORM THE FULL COMPUTE BASELINE ON AVERAGE IN SUCCESS RATE, LATENCY, AND FLOPS.

Model / Benchmark	Version	(V, A, D)	SR (%)	Latency (s)	GFLOPs
SmolVLA / LIBERO	Base	(32, 32, 10)	66.6	0.798	556.6
	Joint	(16, 24, 3)	68.2	0.159	406.3
SmolVLA / Meta-World	Base	(16, 16, 10)	60.7	0.397	318.0
	Joint	(8, 8, 3)	74.9	0.081	173.0
$\pi_{0.5}$ / LIBERO	Base	(18, 18, 10)	87.6	0.537	5000.2
	Joint	(14, 14, 2)	86.7	0.095	3541.0
$\pi_{0.5}$ / Meta-World	Base	(18, 18, 10)	64.9	0.522	5000.2
	Joint	(14, 16, 2)	72.2	0.132	3733.2

B. Single-Axis Trade-offs and Joint Composition (RQ1)

Individual axis roles: In Fig. 3 we analyse the trade-offs between success rate and both latency and FLOPs, for each axis of the compute budget (V, A, D). We observe that the three axes have different effects on the two metrics, with V being the most effective on FLOPs, A on latency and D on both. The number of denoising steps D appears to be a free lever in terms of both latency and FLOPs, usually with a negligible effect on success rate, while sometimes substantially improving performance on Meta-World with $\pi_{0.5}$. This suggests that the optimal number of denoising steps is often not the default one, and that simply reducing it can improve both efficiency and performance, creating a single point on the Pareto curve that dominates the full

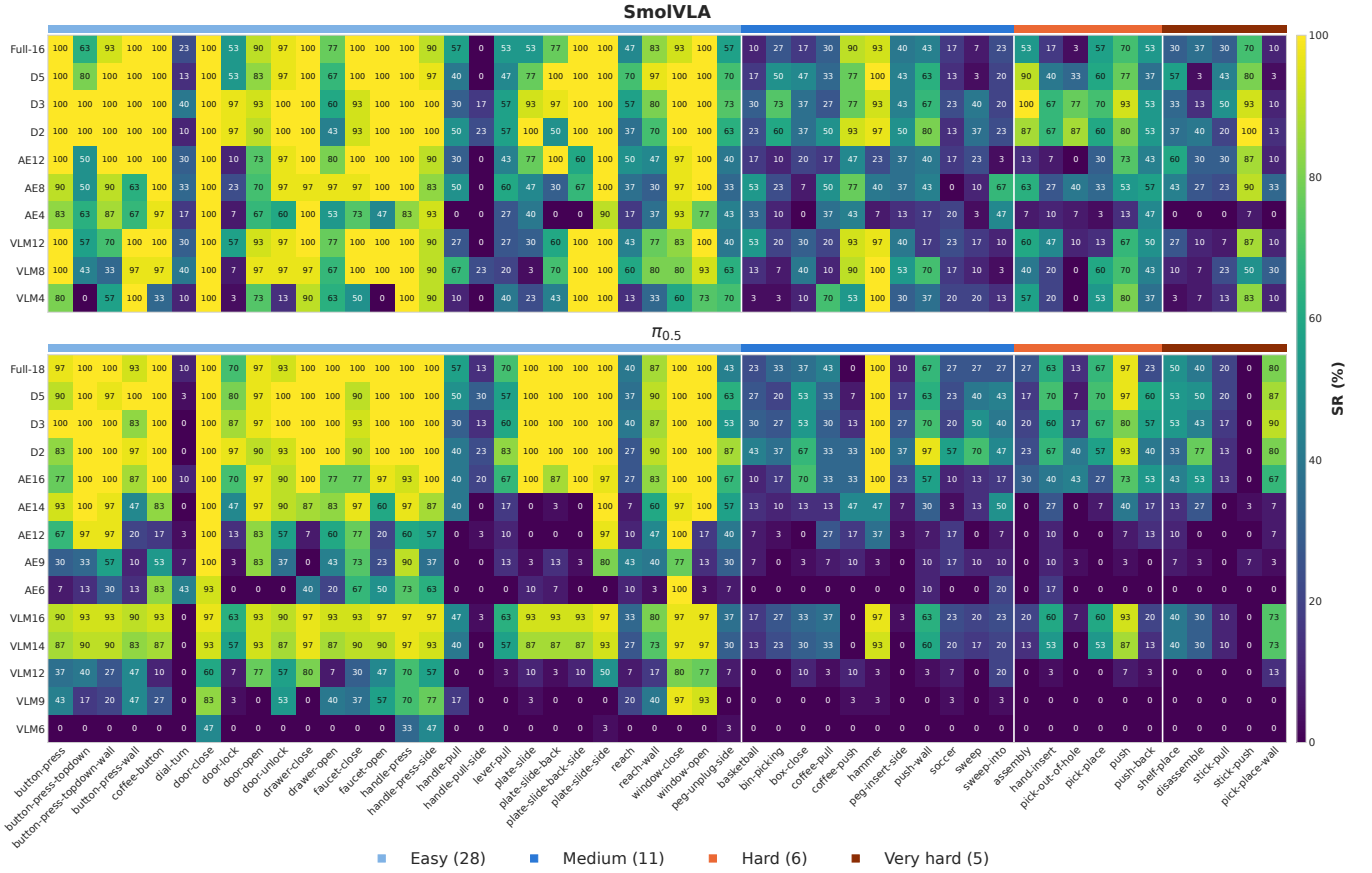


Fig. 4. **Task analysis with the three axes on Meta-World.** Success rate (SR) for SmolVLA (top) and $\pi_{0.5}$ (bottom) across compute budget configurations (rows) and tasks (columns). Rows compare the full baseline (Full) with different compute budgets in (V, A, D) , where only one axis is modified at a time. Each point is averaged over 30 episodes per task. The full policy is optimal on only 11/50 tasks for SmolVLA and 22/50 for $\pi_{0.5}$, with the best configurations distributed across all three axes. Tasks have different requirements and benefit from different axes, motivating task-dependent (V, A, D) .

policy. The action expert early exit A has a strong effect on latency, often halving it when using shallower exits while preserving success rate. On the other hand, the effect on the FLOPs is more contained. Smaller policies such as SmolVLA also have a modest reduction in FLOPs up to approximately 20%, while larger ones such as $\pi_{0.5}$ simply retain the full performance. Regarding the VLM early exit V , we observe substantial FLOPs reductions across both policies and benchmarks. Contrarily to the other two axes, the effect of V on latency is more contained. Finally, this highlights the complementary nature of the three axes, with each one being more effective on a different metric.

Joint configurations: For each model and benchmark pair, we select the operating point with the best trade-off in terms of success and efficiency on each axis and compose the resulting choices of V , A , and D . Table II compares these optimal joint configurations with their corresponding full compute base policies. Across the four settings, the joint configurations reduce latency by 79.2% and FLOPs by 31.8% on average, while improving mean success rate by 5.6%. This confirms that the complementary gains of the three axes can be composed into a single joint configuration that improves the success/efficiency trade-off over the full policy, as well as the corresponding single axis choices.

C. Task-Dependent Compute Requirements (RQ2)

In this section, we investigate the relation between different types of tasks and each axis in our compute budget (V, A, D) . The results in Section IV-B identify one useful operating point, but implicitly assume that every task can save computation from the same part of the policy. We test this assumption with SmolVLA and $\pi_{0.5}$ on Meta-World MT50, whose 50 tasks range across four levels of difficulty. The question is therefore not only *how much* compute a task needs, but *where* to allocate compute in the policy.

A fixed compute budget is suboptimal across tasks:

The full policy is the best configuration on only 11/50 tasks for SmolVLA and 22/50 tasks for $\pi_{0.5}$, with the optimal configuration being spread across the three axes. The selection of the best axis for each task would raise mean success from 60.7% to 79.5% on SmolVLA and from 64.9% to 75.9% on $\pi_{0.5}$. For instance, for task *stick-push* with SmolVLA, the optimal compute configuration for this very hard suite would have been $(8, 8, 3)$, while $(4, 8, 2)$ could achieve even more significant speedups while preserving the success rate. This task-dependent compute budget shows that the compute allocation becomes a choice of *where* to allocate compute across perception V , action generation A ,

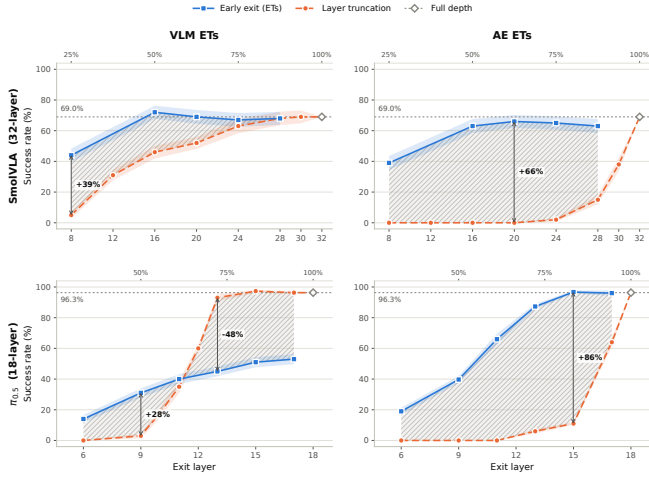


Fig. 5. **Early exits vs. layer truncation.** Average success rate on LIBERO as a function of the exit layer, for the VLM axis V (left) and the action expert axis A (right), on SmolVLA (top) and $\pi_{0.5}$ (bottom). *Layer truncation* reads the action head directly from the intermediate state, while *early exit* routes it through an ET at that depth. The dashed line is the full depth policy; arrows report the gap at the corresponding depth.

and denoising D , rather than only *how much* compute to use.

Tasks are sensitive to different compute axes: The heatmaps in Figure 4 do not degrade uniformly with depth, as a task that can tolerate a shallow exit on one axis can fail on a different one. For example, on SmolVLA, *hammer* retains 100% SR at VLM4 but falls from 93% to 7% at AE4, whereas *faucet-open* retains 100% at AE8 but falls from 100% to 0% at VLM4. In particular, tasks whose goal region tolerates a coarse trajectory (e.g., *door-close*, *drawer-close*, *window-open/close*, *handle-press*) retain a high success rate even with shallow exits in both policies, while tasks that require precise contact collapse (e.g., SmolVLA AE8 on *hammer* 93 $\rightarrow$ 40, *reach-wall* 83 $\rightarrow$ 30; $\pi_{0.5}$ AE14 on *pick-place* 67 $\rightarrow$ 7). This suggests that the action expert depth A matters when that representation must be converted into a precise trajectory, hence for tasks requiring more dexterity. On the other hand, the VLM depth V is more effective when the bottleneck is the visual complexity of the scene. For instance, on SmolVLA, shallow exits are harmless for large and unambiguous targets (e.g., *door-close*, *plate-slide-side* and *handle-press*) that retain 100% with only VLM4, but fatal for small affordances and obstacle variants (e.g., *faucet-open* drops 100 $\rightarrow$ 0, *coffee-button* 100 $\rightarrow$ 33). Finally, among the three axes, the number of denoising steps D is the most robust axis, as it improves the majority of tasks. Specifically, it improves tasks that require more dexterity and where the full policy fails, such as SmolVLA’s Hard group from 42% to 77% (e.g., *pick-out-of-hole* 3 $\rightarrow$ 77 and *assembly* 53 $\rightarrow$ 100).

For a robot executing different manipulation tasks in the real world, compute could potentially be treated as a control variable rather than a fixed property of the policy. The robot can allocate capacity to the component that most benefits the current task, reserving depth in the backbone for complex semantic domains and action expert depth when dexterity

TABLE III

KV CACHE SYNTHESIS ABLATION. REPORTED GAINS OF THE KV CACHE SYNTHESIS MECHANISM OVER THE STANDARD APPROACH ON FULL LIBERO (300 EPISODES PER SUITE FOR SMOLVLA, 400 FOR $\pi_{0.5}$), FOR EACH VLM EARLY EXIT V .

V	SmolVLA (32 layers)			$\pi_{0.5}$ (18 layers)				
	8	16	24	6	9	12	14	16
VLM GFLOPs w/o	86.3	86.3	86.3	3939	3939	3939	3939	3939
VLM GFLOPs w/	21.6	43.1	64.7	1094	1970	2845	3283	3648
Speedup ($\times$)	4.0	2.0	1.3	3.6	2.0	1.4	1.2	1.1
VLA GFLOPs w/o	559	559	559	5213	5213	5213	5213	5213
VLA GFLOPs w/	495	516	538	2368	3244	4119	4557	4894
Speedup (%)	+11.6	+7.7	+3.9	+54.6	+37.8	+21.0	+12.6	+6.1
Δ SR (%)	+0.5	-1.8	+1.8	0.0	0.0	+1.0	-0.5	0.0

is required. Our results establish potential headroom for task-dependent compute allocation, with the online selection of (V, A, D) based on the type of tasks that remains an important direction for future work.

D. Ablation Studies (RQ3)

1) *Exit Transformers vs. Layer Truncation:* The simplest way to reduce depth is layer truncation, which stops the forward pass at an intermediate layer and uses the resulting representation without additional parameters or training. We compare our ETs against this baseline to assess whether trained exits improve success rate enough to justify their additional parameters and computation, particularly at shallow depths. In our implementation of layer truncation, the action head reads directly from the intermediate layer representation without an ET. Fig. 5 shows the results of this comparison. In the action expert, the early exits consistently outperform layer truncation, especially at shallower depths, where the gap can be as large as 86% in success rate. Regarding the VLM, we notice the same results on SmolVLA, while for $\pi_{0.5}$ we observe a different trend: early exits perform better at shallow depths, but at deeper ones $\pi_{0.5}$ recovers and outperforms the early exits. We attribute this to the larger scale of $\pi_{0.5}$, whose intermediate representations are already informative and appear to be sufficient to condition the action head without the need for an ET. Consistently, in the action expert, where the size gap between the two models is smaller, the ETs are effective at all evaluated depths in both models.

2) *KV Cache Synthesis Ablation:* In this section, we aim to isolate the effect of our KV cache synthesis mechanism. Table III illustrates the gains on LIBERO with SmolVLA and $\pi_{0.5}$, for each VLM early exit V selected in Table I. On both VLAs, the gains are substantial and correlated with the number of skipped layers, with a $4\times$ speedup on SmolVLA and up to $3.6\times$ on $\pi_{0.5}$. Notably, as demonstrated by the negligible variation in Δ SR, the addition of the KV cache synthesis mechanism does not affect the success rate. Furthermore, we observe two different trends in the compute savings. For instance, on SmolVLA the prefix is a small part of the total compute (86.3 of 559 GFLOPs), so even a $4\times$ prefix reduction is worth 11.6% at the end. On the other hand, the prefix dominates on $\pi_{0.5}$ (3939 of 5213

GFLOPs) and the same mechanism removes up to 54.6% of the total compute. The synthesis is therefore the component that makes the V axis a compute lever on large backbones.

V. CONCLUSIONS

We presented a training framework for pretrained flow-matching VLAs that allows us to study the depth (i.e., the number of layers) of the VLM backbone V , the depth of the action expert A , and the number of denoising steps D as three jointly configurable compute axes. Exit Transformers distilled from the frozen policy make intermediate depths valid exits in the original action space, and a KV cache synthesis mechanism fills the prefix cache of the skipped backbone layers, allowing the action expert to exit deeper than the backbone. Across two policies (i.e., SmolVLA and $\pi_{0.5}$) and two benchmarks (i.e., LIBERO and Meta-World), the three axes proved complementary: V reduces FLOPs, A affects latency, and D improves both, with their joint configuration dominating the full policy and the single axis on the Pareto frontier. Our task analysis further revealed that no fixed compute budget is optimal across tasks and that each task is sensitive to different compute axis in (V, A, D) , with intermediate distilled representations potentially outperforming the full model.

Nevertheless, some limitations remain. For instance, early exits are attached at specific depths, leaving open the question of what the optimal set of depths is for each policy.

Finally, as showcased by our task analysis, VLAs should learn to flexibly allocate compute budget to improve efficiency and performance. Future work should investigate adaptive compute selection strategies, able to tailor compute allocation to each timestep of an episode or across task types, with the potential to further improve the Pareto frontier.

REFERENCES

- [1] K. Kawaharazuka, J. Oh, J. Yamada, I. Posner, and Y. Zhu, "Vision-language-action models for robotics: A review towards real-world applications," *IEEE Access*, 2025.
- [2] X. Yao, H. Zhou, O. Mees, Y. Meng, T. Xiao *et al.*, "Bridging language and action: A survey of language-conditioned robot manipulation," *The International Journal of Robotics Research*, 2026.
- [3] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow matching for generative modeling," in *The Eleventh International Conference on Learning Representations*, 2023. [Online]. Available: <https://openreview.net/forum?id=PqvMRDCJT9t>
- [4] T. Yu, D. Quillen, Z. He, R. Julian, K. Hausman *et al.*, "Meta-world: A benchmark and evaluation for multi-task and meta reinforcement learning," in *Conference on robot learning*. PMLR, 2020, pp. 1094–1100.
- [5] Y. Yang, Y. Wang, Z. Wen, L. Zhongwei, C. Zou *et al.*, "Efficientvla: Training-free acceleration and compression for vision-language-action models," *Advances in Neural Information Processing Systems*, vol. 38, pp. 40 891–40 914, 2026.
- [6] R. Zhang, M. Dong, Y. Zhang, L. Heng, X. Chi *et al.*, "Mole-vla: Dynamic layer-skipping vision language action model via mixture-of-layers for efficient robot manipulation," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 40, no. 22, 2026, pp. 18 764–18 772.
- [7] Y. Yue, Y. Wang, B. Kang, Y. Han, S. Wang *et al.*, "Deer-vla: Dynamic inference of multimodal large language models for efficient robot execution," *Advances in Neural Information Processing Systems*, vol. 37, pp. 56 619–56 643, 2024.
- [8] K. Zhang, J. Zhang, R. Xu, Y. Sun, S. Xue *et al.*, "A1: A fully transparent open-source, adaptive and efficient truncated vision-language-action model," *arXiv preprint arXiv:2604.05672*, 2026.
- [9] S. Xu, Y. Wang, C. Xia, D. Zhu, T. Huang *et al.*, "Vla-cache: Efficient vision-language-action manipulation via adaptive token caching," *Advances in Neural Information Processing Systems*, vol. 38, pp. 164 448–164 473, 2026.
- [10] W. Luan, J. Li, W. Zhao, W. Zhang, T. Wu *et al.*, "Snapflow: One-step action generation for flow-matching vlars via progressive self-distillation," *arXiv preprint arXiv:2604.05656*, 2026.
- [11] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao *et al.*, "Rt-2: Vision-language-action models transfer web knowledge to robotic control," in *Proceedings of The 7th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, J. Tan, M. Toussaint, and K. Darvish, Eds., vol. 229. PMLR, 06–09 Nov 2023, pp. 2165–2183.
- [12] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna *et al.*, "Openvla: An open-source vision-language-action model," in *Proceedings of The 8th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, P. Agrawal, O. Kroemer, and W. Burgard, Eds., vol. 270. PMLR, 06–09 Nov 2025, pp. 2679–2713.
- [13] K. Black, N. Brown, D. Driess, A. Esmail, M. R. Equi *et al.*, " π_0 : A vision-language-action flow model for general robot control," in *RSS 2025*, 2025.
- [14] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma *et al.*, "Smolvla: A vision-language-action model for affordable and efficient robotics," *arXiv preprint arXiv:2506.01844*, 2025.
- [15] J. Wen, Y. Zhu, J. Li, M. Zhu, Z. Tang *et al.*, "Tinyvla: toward fast, data-efficient vision-language-action models for robotic manipulation," *IEEE Robotics and Automation Letters*, vol. 10, no. 4, pp. 3988–3995, 2025.
- [16] S. Laskaridis, A. Kouris, and N. D. Lane, "Adaptive inference through early-exit networks: Design, challenges and directions," in *Proceedings of the 5th International Workshop on Embedded and Mobile Deep Learning*, 2021, pp. 1–6.
- [17] S. Teerapittayanon, B. McDanel, and H.-T. Kung, "Branchynet: Fast inference via early exiting from deep neural networks," in *2016 23rd international conference on pattern recognition (ICPR)*. IEEE, 2016, pp. 2464–2469.
- [18] J. Xin, R. Tang, J. Lee, Y. Yu, and J. Lin, "Deebert: Dynamic early exiting for accelerating bert inference," in *Proceedings of the 58th annual meeting of the association for computational linguistics*, 2020, pp. 2246–2251.
- [19] W. Yu, T. Wang, F. Li, J. Li, and L. Zhu, "Ac²-vla: Action-context-aware adaptive computation in vision-language-action models for efficient robotic manipulation," *arXiv preprint arXiv:2601.19634*, 2026.
- [20] W. Song, J. Chen, P. Ding, Y. Huang, H. Zhao *et al.*, "Ceet-vla: Consistency vision-language-action model with early-exit decoding," *arXiv preprint arXiv:2506.13725*, 2025.
- [21] K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess *et al.*, " $\pi_{0.5}$: a vision-language-action model with open-world generalization," in *Proceedings of The 9th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, J. Lim, S. Song, and H.-W. Park, Eds., vol. 305. PMLR, 27–30 Sep 2025, pp. 17–40.
- [22] D. J. Bajpai and M. K. Hanawal, "Free: fast and robust vision language models with early exits," in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025, pp. 23 599–23 615.
- [23] M. Elbayad, J. Gu, E. Grave, and M. Auli, "Depth-adaptive transformer," in *International Conference on Learning Representations*, 2020. [Online]. Available: <https://openreview.net/forum?id=SJg7KhVKPH>
- [24] T. Schuster, A. Fisch, J. Gupta, M. Dehghani, D. Bahri *et al.*, "Confident adaptive language modeling," *Advances in Neural Information Processing Systems*, vol. 35, pp. 17 456–17 472, 2022.
- [25] M. Elhoushi, A. Shrivastava, D. Liskovich, B. Hosmer, B. Wasti *et al.*, "Layerskip: Enabling early exit inference and self-speculative decoding," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 12 622–12 642.
- [26] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu *et al.*, "Libero: Benchmarking knowledge transfer for lifelong robot learning," *Advances in Neural Information Processing Systems*, vol. 36, pp. 44 776–44 791, 2023.
- [27] R. Cadene, S. Alibert, F. Capuano, M. Aractingi, A. Zouitine *et al.*, "Lerobot: An open-source library for end-to-end robot learning," in *International Conference on Learning Representations*, vol. 2026, 2026, pp. 122 398–122 417.